به نام خدا

ساختمان داده‌ها و الگوریتم‌ها (۴۰۲۵۴)

دانشگاه صنعتی شریف — تمرین دوم

مدرس: مهدی صفرنژاد — انتشار: ۱۲ اسفند ۹۸

داده‌ساختارهای پایه، درخت‌ها

سؤال ۱. آ. می‌دانیم که پشته‌ها FILO و صف‌ها FIFO هستند. حال فرض کنید که دو پشته داریم که پشته‌ی اول پر و پشته‌ی دوم خالی می‌باشد. حال به ترتیب اجزای پشته‌ی اول را خوانده و در پشته‌ی دوم می‌ریزیم، چون پشته‌ها FILO هستند، اجزایی که زودتر وارد پشته‌ی اول شده‌اند دیرتر وارد پشته‌ی دوم می‌شوند. چون پشته‌ی دوم نیز FILO است، اجزایی که دیرتر وارد شده‌اند زودتر خارج می‌شوند، پس هر عضوی که زودتر وارد پشته‌ی اول شده باشد، بعد از خالی کردن پشته‌ی اول به پشته‌ی دوم زودتر خارج می‌شود که مانند یک FIFO می باشد. حال با کمک این ایده به کمک دو پشته یک صف را طراحی می‌کنیم:

هنگام push به پشته‌ی اول push می‌کنیم. هنگام pop نیز از پشته‌ی دوم pop میکنیم، اما اگر پشته‌ی دوم خالی بود، ابتدا تمام اعضای پشته‌ی اول را وارد پشته‌ی دوم می‌کنیم. سپس از پشته‌ی دوم pop میکنیم. عملیات push چون عملیات push به پشته است از $O(1)$ است. عملیات pop شامل حداکثر n عمل pop برای خالی کردن پشته‌ی اول n عمل push برای وارد شدن به پشته‌ی دوم و یک عملیات pop از پشته‌ی دوم می‌باشد. پس از $O(n)$ می‌باشد. (گر چه پس از یادگیری تحلیل سرشکن می‌توانید بررسی کنید که مرتبه‌ی زمانی pop نیز به طور سرشکن از $O(1)$ می‌باشد چون هر عضو حداکثر دو بار pop می‌شود.)

کد مربوط به این الگوریتم در ادامه آمده‌است.

```
Queue:
        def push(element):
                stack1.push(element)

        def pop():
                if not stack2.empty():
                        return stack2.pop()
                while not stack1.empty():
                        stack2.push(stack1.pop())
                return stack2.pop()
```

ب. در این قسمت از دو هرم برای خواسته‌ی مسئله استفاده می‌کنیم، ایده‌ی ما به این صورت است که نیمه‌ی بیشتر اعضا را در یک هرم کمینه و نیمه‌ی کمتر اعضا را در یک هرم بیشینه نگه‌داری نماییم. این ساختار این مزیت را به ما می‌دهد که همیشه میانه در یکی از دو ریشه‌ی این هرم‌ها می‌باشد و با $O(1)$ قابل دسترسی می‌باشد. حال مسئله این است که چگونه باید عملیات insert را در $O(\log(n))$ انجام دهیم به طوری که دو پشته‌ی با اختلاف اندازه‌ی حداکثر برابر یک داشته باشیم.

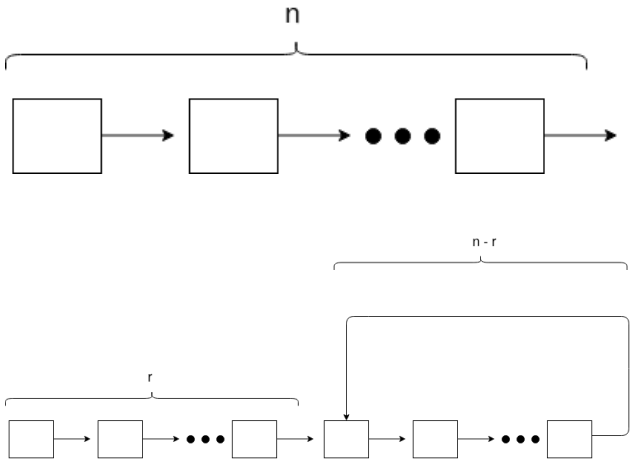
برای انجام اینکار کافیست insert را بگونه‌ای انجام دهیم که اگر دو پشته خاصیت بالا را داشتند، بعد insert نیز این خاصیت حفظ شود. چون که حالت ابتدایی داده ساختار نیز با دو پشته‌ی خالی که خاصیت بالا را دارند شروع می‌گردد، همیشه این خاصیت را خواهند داشت.

برای insert ابتدا عضو جدید را با ریشه‌ی دو هرم مقایسه میکنیم.اگر بیشتر از ریشه‌ی هرم کمینه باشد، آن را در هرم کمینه، اگر کمتر از ریشه‌ی هرم بیشینه بود آن را در هرم بیشینه و اگر بین این دو مقدار بود، آن را در یکی از هرم‌های با اندازه‌ی کمتر قرار می‌دهیم. توجه کنید که چون نیمه‌ی بیشتر اعداد در هرم کمینه و نیمه‌ی کمتر در هرم بیشینه بودند، ریشه‌ی هرم کمینه بیشتر از ریشه‌ی هرم بیشینه می‌باشد. پس تنها یکی از سه حالت بالا می‌توانست رخ بدهد. همچنین با روش بالا، خاصیت بیشتر بودن هرم کمینه نسبت به بیشینه همچنان حفظ می‌گردد. برای مقایسه‌ها و دسترسی به ریشه‌های دو هرم و قرار دادن اجزا در هرم مجموعا ($O(\log(n))$) هزینه دارد.

حال پس از وارد شدن عضو جدید به یک از دو هرم، اندازه‌ی دو هرم حداکثر به اندازه‌ی دو واحد تفاوت خواهد داشت. اگر این تفاوت به اندازه‌ی یک واحد باشد، به منظور خود رسیده‌ایم اما فرض کنید که این تفاوت اندازه دو واحد باشد، هرم با اندازه‌ی بیشتر را در نظر بگیرید، می‌توان عضو ریشه‌ی این هرم را جدا کرده و آن را به هرم دیگر وارد کرد، با انجام این کار چون که ریشه‌های هرم ها از تمام اجزای هرم بزرگتر یا کوچکتر می‌باشند، هنوز هم درخت کمینه بیشتر از درخت بیشینه می‌باشد. همچنین اندازه ی دو هرم نیز برابر می‌شود و انجام این کار نیز مرتبه‌ی زمانی insert را از ($O(\log(n))$) تغییر نخواهد داد.

سؤال ۲. الگوریتمی از ($O(n^2)$) برای مسئله پیشنهاد می‌دهیم.

اندازه‌ی لیست پیوندی را n در نظر بگیرید. این لیست به یکی از دو فرم زیر می‌باشد.



برای تشخیص وجود دور از دو پوینتر استفاده میکنیم. در هر مرحله پوینتر اول یک واحد و پوینتر دوم دو واحد جلو می‌رود. در صورتی که نتوانند این عملیات را انجام دهند، به این معنی است که لیست پیوندی دور ندارد و الگوریتم پایان می‌یابد. اگر در یک مرحله پوینتر اول با پوینتر دوم برابر شود به این معنی است که لیست پیوندی دور دارد و الگوریتم پایان می‌یابد.

اثبات درستی الگوریتم:

اگر لیست پیوندی دور نداشته باشد، پس از $\lceil \frac{n}{۲} \rceil$ حرکت پوینتر دوم به انتهای لیست پیوندی میرسد و الگوریتم با نتیجه‌ی درست خارج می‌شود. (به راحتی می‌توانید اثبات کنید که در صورتی که در یک مرحله پوینتر اول برابر پوینتر دوم شود، با فرض دور نداشتن لیست تناقض دارد)

اگر لیست دور داشته باشد، چون همیشه هر دو پوینتر می‌توانند به عنصر بعدی بروند، هیچگاه الگوریتم با نتیجه‌ی دور نداشتن خارج نمی‌شود. برای پایان‌پذیری و درست الگوریتم در این حالت کافیست توجه کنیم که اگر الگوریتم $n(n-r)$ مرحله ادامه پیدا کند، پوینتر اول درون دور قرار دارد($n(n-r) > r$). پوینتر دوم با پوینتر اول به اندازه‌ی $n(n-r)$ واحد اختلاف دارد. که مضربی از اندازه‌ی دور می‌باشد. پس انگار که پوینتر دوم، همان پوینتر اول است تنها n بار بیشتر دور را پیموده است. (با نوشتن روابط همنهشتی می‌توانید اثبات دقیق‌تری برای خود بنویسید.) پس در حداکثر $n(n-r)$ مرحله از $O(۱)$ الگوریتم با جواب صحیح پایان می‌یابد که از $O(n^۲)$ می‌باشد.

سؤال ۳. قسمت اول سوال یک قسمت کمکی برای قسمت دوم بوده‌است. راه‌حل این سوال به صورت کامل در این لینک توضیح داده‌شده‌است.

سؤال ۴. داده ساختار اصلی یک پشته برای عملیات‌های push و pop است. برای یافتن min نیز از یک پشته‌ی کمکی هم ارتفاع با پشته‌ی اصلی استفاده می‌کنیم که در ارتفاع hم آن، کمینه‌ی h عنصر اول پشته‌ی اصلی را نگه می‌داریم. برای مدیریت این استک نیز به صورت زیر عمل می‌کنیم:

هنگام pop از استک اصلی، از این استک نیز pop می‌کنیم. هنگام push کردن به استک اصلی، عضو جدید را با اخرین عضو پشته کمکی مقایسه می‌کنیم. اگر کوچکتر بود، عضو جدید وگرنه آخرین عضو پشته‌ی کمکی را به پشته‌ی کمکی push می‌کنیم. از آنجایی که عضو انتهایی کمینه‌ی سایر اعضا تا به اینجا بوده‌است، با روشی که ارائه کردیم، عضو جدید پشته‌ی کمکی، کمینه‌ی اعضای پشته‌ی اصلی می‌باشد.

برای deleteMin، نمی‌توان این کار را انجام داد. برای اثبات فرض کنید که بشود deleteMin را در مرتبه‌ی $O(۱)$ انجام داد. در این صورت تمام اعضای یک ارایه را در $O(n)$ درون این داده ساختار می‌ریزیم سپس یکی یکی کمینه را یافته در یک آرایه‌ی جدید ریخته و کمینه را از داده‌ساختار فعلی حذف کند تا داده ساختار خالی شود و هر کدام از این عملیات‌ها در $O(۱)$ و در کل در $O(n)$ انجام می‌شود. در این صورت توانسته‌ایم که یک آرایه‌ی دلخواه را در زمان $O(n)$ مرتب کنیم که طبق قضایا می‌دانیم که مرتب‌سازی آرایه حداقل از مرتبه‌ی زمانی $\Omega(n\log(n))$ می‌باشد و این تناقض می‌باشد.

سؤال ۵. حداکثر ۸تا

زیرگراف شامل اعداد ۱۰۰۱ تا ۱۰۲۴ را در نظر بگیرید، چون که هرم یک هرم کمینه است، و ۲۴ عدد بزرگترین اعداد این هرم می‌باشند، زیرگراف حاصل همبند و شامل ریشه می‌باشد پس خود یک درخت مانند درخت کمینه بدون شرط نقریبا کامل بودن می‌باشد. این گراف را T می‌نامیم.

لم: اگر در ارتفاع h از یک درخت دودویی k رأس باشد در ارتفاع $h-۱$ حداقل $\lceil \frac{k}{۲} \rceil$ رأس می‌باشد.

اثبات: فرض کنید که لم برقرار نباشد. هر رأس از ارتفاع h یک پدر از ارتفاع $h-۱$ دارد، لذا اگر رأس‌های این دسته را بر حسب پدرهایشان دسته‌بندی کنیم، طبق اصل لانه‌ی کبوتری حداقل یک دسته با بیش از سه عضو وجود دارد که مخالف فرض دودویی بودن درخت می‌باشد.

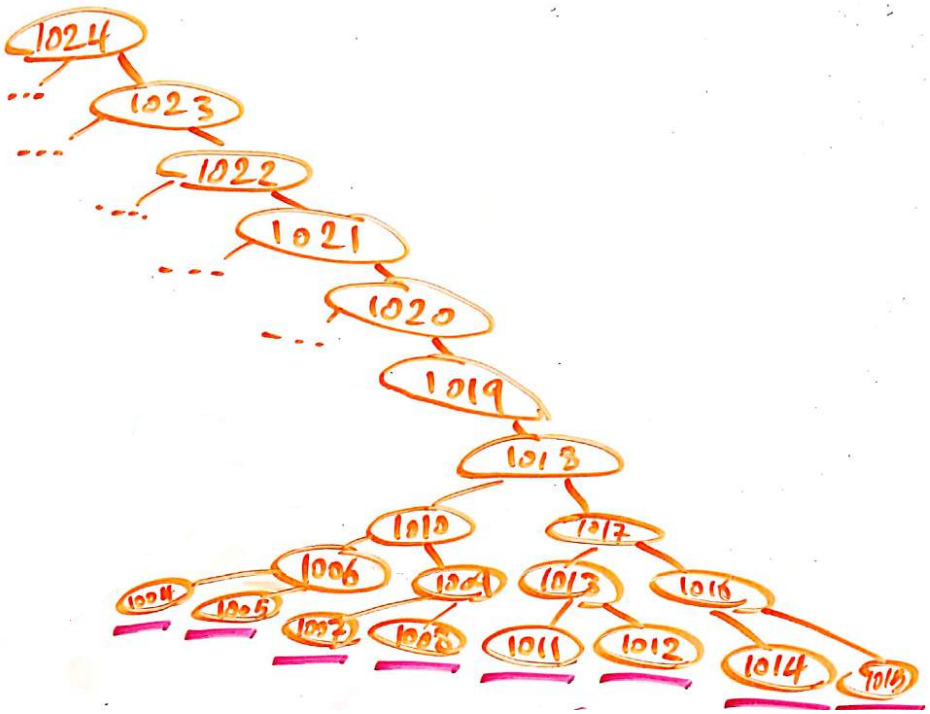
چون که $۲^{۱۰} = ۱۰۲۴$ با کنار گذاشتن یک رأس درخت تبدیل به یک درخت کامل با ارتفاع ۱۰ می‌شود. پس همه‌ی برگ‌های درخت غیر از یکی ارتفاع ۱۰ دارند.

حال درخت T را در نظر بگیرید، هدف بیشینه کردن تعداد برگ‌های این درخت در عمق ۱۰ می‌باشد.

ادعا می‌کنیم که این تعداد کمتر از ۹ واحد می‌باشد.

اگر این تعداد حداقل ۹ واحد باشد، در ارتفاع‌های ۱ تا ۱۰ به ترتیب حداقل $[۱, ۱, ۱, ۱, ۱, ۱, ۲, ۳, ۵, ۹]$ رأس باید موجود باشد. یعنی حداقل ۲۵ رأس.

برای ۸ رأس مثال زیر از بین پاسخ‌ها انتخاب شده‌است. (با تشکر از محمدصدرا حیدری)



سؤال ۶. در الگوریتم merge کردن دو دنباله‌ی مرتب دو عضوی از دو آرایه که اشاره‌گر روی آن‌ها بود را مقایسه می‌کردیم و عضو کمتر را به آرایه‌ی جواب اضافه کرده و اشاره‌گر را در این آرایه‌ی عضو کمتر به جلو می‌بریم. حال به جای دو دنباله، d دنباله از اعداد مرتب‌شده را داریم و این مقایسه باید بین عضوهای دارای اشاره‌گر همه‌ی دنباله‌ها انجام شود. برای اینکار از یک هرم کمینه استفاده می‌کنیم. در ابتدا d عضو اول این d دنباله را در $O(d)$ در یک هرم کمینه قرار می‌دهیم. در هر مرحله کمینه‌ی اعضا را با $O(\log(d))$ از هرم خارج کرده و در آرایه‌ی جواب می‌گذاریم، سپس اشاره‌گر را در ارایه‌ی شامل عضو کمینه جلو می‌بریم و اگر تمام نشده بود عضو جدید را با $O(\log(d))$ به هرم کمینه اضافه می‌کنیم.

توجه کنید که برای تشخیص اینکه عضو کمینه متعلق به کدام آرایه است، اعضا را به صورت جفت مرتب مقدار عضو و اندیس آرایه‌ای که عضو به آن متعلق است به هرم کمینه اضافه می‌کنیم تا هنگام به دست آمدن عضو کمینه، اندیس آرایه‌ی مربوط در بین d آرایه را نیز به دست آوریم.

با یک هزینه‌ی $O(d)$ اولیه و n بار اضافه شدن و کم شدن از هرم کمینه با هزینه‌ی $O(\log(d))$ در مجموع الگوریتم در مرتبه‌ی $O(n\log(d))$ انجام می‌شود.

سؤال ۷. ابتدا در مرتبه‌ی $O(n)$ تمام اعضای آرایه را در یک هرم بیشینه ذخیره می‌کنیم. سپس به تعداد k بار عضو کمینه را از هرم با $O(\log(n))$ حذف می‌کنیم، بدین صورت k عضو بزرگتر در مرتبه‌ی زمانی خواسته‌شده به دست می‌آیند.

خواسته‌ی سوال را در مرتبه‌ی زمانی $O(n)$ نیز می‌توان به دست‌آورد. به کمک روش medians، of median ابتدا kامین عنصر بزرگتر را می‌یابیم، سپس با یک گذر از روی آرایه اعضای بزرگتر مساوی این عضو را بر می‌داریم.

روش بالا ممکن است بیشتر از k عضو به ما معرفی کند. برای حل این مشکل با کمی تغییر در روش بالا و نگه داشتن کمترین این اعضا در هر مرحله، می‌توانید این تعداد را به k واحد کاهش دهید.

سؤال ۸. ابتدا مسیر ریشه تا a را در یک آرایه مانند A ذخیره می‌کنیم. سپس همین کار را در مورد راس b و آرایه‌ی B انجام می‌دهیم.

$$B = [root...b]\ A = [root...a]$$

از ابتدای این دو آرایه شروع کرده و اخرین راس مشترک را پیدا میکنیم و آن را c نامگذاری می‌کنیم. c همان پدر مشترک این دو راس می‌باشد. ادامه‌ی آرایه از c در A و B مسیر بین این دو راس را تشکیل می‌دهد که به راحتی میتوان بیشینه را در بین این دو آرایه یافت.

سؤال ۹. تعریف فرضی: چپ‌ترین نوه را برای یک راس مانند v راسی میگوییم که از به بچه‌ی چپ رفتن متوالی با شروع از راس v به دست می‌آید.

به راحتی می‌توان مشاهده کرد که چپ‌ترین نوه‌ی v، کوچکترین راس در زیردرخت به ریشه‌ی v می‌باشد.

توضیح الگوریتم روی رأس v: اگر v فرزند راست داشت جواب چپ‌ترین نوه‌ی فرزند راست vمی‌باشد. چون که فرزندان چپ v از آن کوچکترند و سایر اجزای درخت نیز یا از کل رئوس زیردرخت v کوچکترند یا بزرگتر.

در صورت برقرار نبودن شرط بالا، با شروع از راس v برای یک رأس در صورتی که فرزند راست پدرش باشد به پدرش می‌رویم. هنگامی که یک رأس فرزند چپ پدرش بود، پدر آن رأس اولین عضو بزرگ بعد v خواهد بود. اگر روش بالا جواب نداد حتما در بزرگترین رأس درخت بوده‌ایم.

تحلیل زمانی:

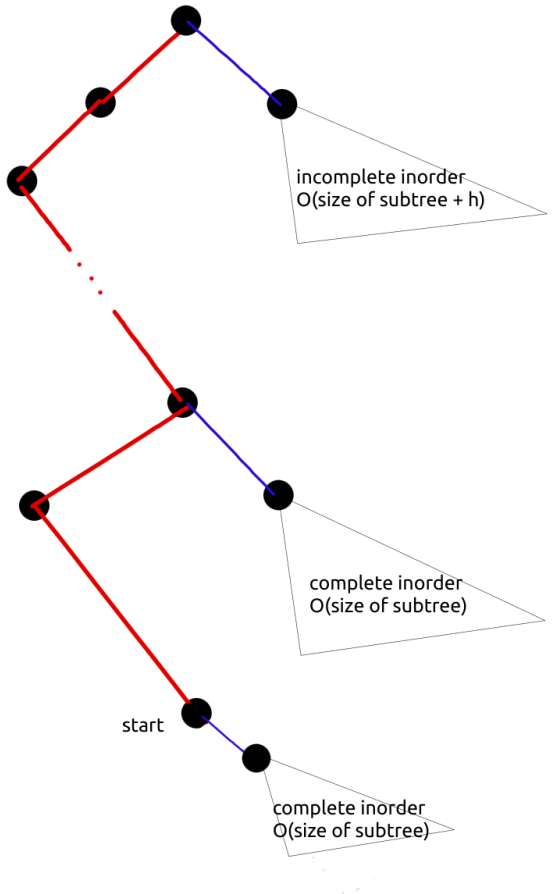
فرض کنید که k از اندازه‌ی زیردرخت فرزند راست رأس v نابیشتر باشد. در این حالت الگوریتم مانند یک پیمایش inorder روی فرزند راست رأس v عمل می کند و می‌دانیم که به دست آوردن k عنصر اول inorder از مرتبه‌ی $O(h+k)$ می‌باشد. (به راحتی میتوانید به صورت تفکر بازگشتی و به کم استقرا این موضوع را بررسی نمایید)

حال فرض کنید که k از اندازه‌ی زیردرخت فرزند راست رأس v بیشتر باشد. در این صورت پس از پیمایش inorder کل زیردرخت راست v در مرتبه‌ی زمانی خطی نسبت به اندازه‌ی آن، حالت دوم الگوریتم رخ داده و شروع به رفتن به پدر می‌کنیم تا به پدری برسیم که از فرزند چپ آن وارد شده باشیم. سپس برای این راس ابتدا زیردرخت راست آن‌را (در حالت تهی بودن یک زیردرخت تهی) را می‌پیماییم و دوباره به پدران رئوس می‌رویم، مشاهده می‌شود که رئوسی که فرزند راست آن‌ها باز می‌شود تشکیل یک مسیر از پایین درخت به بالای آن را می‌دهند. و هر راسی که سمت راست آن باز میشود پدر راس‌های قبلی با این ویژگی بود است.

طول این مسیر از $O(h)$ است و تنها یک بار از پایین به بالا پیموده می‌شود، بقیه‌ی هزینه مربوط به پیمایش inorder فرزندان راست رئوس این مسیر می‌باشد که هزینه‌ی آن برابر اندازه‌ی زیردرخت می‌باشد و به اندازه‌ی زیردرخت از k تا مصرف می‌کند.

پس زمان الگوریتم از مرتبه‌ی طول مسیر که حداکثر $O(h)$ و مرتبه‌ی مجموع اندازه‌ی زیر درخت‌ها برای پیمایش inorder آن‌ها به علاوه‌ی یک h برای پیمایش ناقص زیردرخت آخر یعنی از مرتبه‌ی $O(h+k)$ می‌باشد و مجموعا $O(h+k($ می‌باشد.

شکل زیر شهود بیشتری درباره‌ی چگونگی پیمایش مسیر توسط الگوریتم و تحلیل زمانی به شما می‌دهد.



سؤال ۱۰. اگر از بالای ساختمانی با ارتفاع (H) به سمت راست لیزر زده شود، برای هر کدام از ساختمان‌های با ارتفاع بیشتر از (H) مانند (h)، به اندازه‌ی (h - H) خرابی به بار می آید. اگر جمع این مقدارها را برای هر ساختمان بدست بیاوریم حاصل برابر خواهد شد با : (S - c * H) که (S) جمع ارتفاع ساختمان‌های بلندتر از (H) است که در سمت راست ساختمان با ارتفاع (H) قرار دارند.

بنابر این در ابتدا با درخت AVL دو مقدار را برای هر کدام از ساختمان‌ها بدست می آوریم : جمع ارتفاع ساختمان‌های بلندتر که در سمت راست این ساختمان قرار دارند(sum[i]) و تعداد ساختمان‌های بلندتر با این ویژگی (count[i]).

برای این کار یک درخت AVL داریم که در ابتدا خالی است و به ترتیب از راست به چپ اعداد را به این درخت اضافه می‌کنیم و در طول عملیات insert دو مورد گفته شده را هم بدست می‌آوریم. به این شکل که برای هر راس در این درخت دو مقدار را نگه می‌داریم: اول جمع اعداد زیر درخت آن (S[i]) و دوم تعداد رئوس زیر درخت آن راس (C[i]). حال در عملیات insert یک راس جدید از ریشه شروع می‌کنیم و اگر از راسی که روی آن هستیم بزرگتر بود به فرزند راست می‌رویم و در غیر این صورت به فرزند چپ (همانطور که در insert عادی عمل می‌کنیم). در هر مرحله اگر روی راس i باشیم و بخواهیم راس j را به درخت اضافه کنیم، اگر به فرزند چپ i رفتیم یعنی h[i] > h[j]، در این صورت تمام زیردرخت فرزند راست راس i از راس j مقدار بیشتری دارند و چون از راست به چپ مقادیر ارتفاع ساختمان‌ها را وارد کردیم همه‌ی آن‌ها سمت راست ساختمان j قرار دارند. پس sum[j] را با s[i.right] و count[j] را با c[i.right] جمع می‌زنیم (به علاوه‌ی مقدار خود راس i). اگر هم به فرزند راست i رفتیم کاری نمیکنیم. در پایان عملیات insert هم درخت را متوازن می‌کنیم و در طی آن مقادیر c[i] و s[i] را برای رئوسی که زیردرختشان تغییر کرده update می‌کنیم (از برگ ها شروع کرده و هرکس جمع دو فرزندش + مقدار خودش می‌شود).

حال باید برای هر ساختمان اولین ساختمان بلندتر یا هم ارتفاع سمت راست آن را بدست بیاوریم (r[i]). برای این منظور یک پشته‌ی صعودی در نظر می‌گیریم که در هر لحظه ساختمان‌هایی که می‌توانند کاندید اولین ساختمان بلندتر سمت راست ساختمانی باشند در آن قرار دارند. در ابتدا پشته خالی است. از سمت راست شروع می‌کنیم و به هر ساختمانی که رسیدیم، تا وقتی که ارتفاع ساختمان از ساختمان سر پشته بیشتر بود سر پشته را حذف می‌کنیم چرا که ساختمان سر پشته برای ساختمان‌های چپ‌تر از این ساختمان نمی‌تواند کاندید باشد. دلیل آن هم این است که اگر نوبت ساختمان iام باشد و ساختمان سر پشته ساختمان jام باشد، می‌دانیم h[i] < h[j] و i < j پس اگر ساختمان j بخواهد اولین ساختمان بلندتر سمت راست ساختمانی مثل k باشد، چون ارتفاع ساختمان i بیشتر است و چپ‌تر از ساختمان j هم هست، پس ساختمان i هم از ساختمان k بلندتر است و راست آن ساختمان است و قبل از ساختمان j هم قرار دارد که تناقض است چرا که ساختمان i باید اولین ساختمان بلندتر سمت راست میبوده. بنابر این عملیات حذف از سر پشته را ادامه می‌دهیم تا به یک ساختمان بلندتر برسیم. اولین جاییکه به ساختمان بلندتر از ساختمان iام در پشته برسیم، همان ساختمان r[i] مورد نظر است (یعنی اولین ساختمان بلندتر از ساختمان i که راست آن است) چرا که در بین کاندیدا ها زودتر از همه در پشته به آن رسیدیم یعنی هم ارتفاعش بلندتر است و هم سمت چپ کاندیداهای دیگر است پس اولین ساختمان بلندتر سمت راست ساختمان iام است. سپس ساختمان i را به روی پشته اضافه می‌کنیم.

کد نمونه (با تشکر از مهدی جعفری) :

```python
from sys import stdin, stdout


class Node:
    def __init__(self, val: int):
        self.key = val
```

```
        self.left = None
        self.right = None
        self.h = 1
        self.sum = val
        self.size = 1


tmp = []


class Tree:

    def __init__(self):
        self.root = None

    def insert(self, val: int):
        self.root = self.Insert(self.root, val)

    def Insert(self, root, val: int):
        if root is None:
            return Node(val)
        if val <= root.key:
            global tmp
            tmp[0] += Tree.getSum(root.right) + root.key
            tmp[1] += Tree.getSize(root.right) + 1
            root.left = self.Insert(root.left, val)
        else:
            root.right = self.Insert(root.right, val)

        Tree.calc(root)
        balance = Tree.getHeight(root.left) - Tree.getHeight(root.right)
        if balance > 1:
            if val <= root.left.key:
                return Tree.rightRotate(root)
            else:
                root.left = Tree.leftRotate(root.left)
                return Tree.rightRotate(root)
        elif balance < -1:
            if val <= root.right.key:
                root.right = Tree.rightRotate(root.right)
                return Tree.leftRotate(root)
            else:
                return Tree.leftRotate(root)
```

```
50          else:
51              return root
52
53      @staticmethod
54      def rightRotate(y):
55          x = y.left
56          y.left = x.right
57          x.right = y
58
59          Tree.calc(y)
60          Tree.calc(x)
61          return x
62
63      @staticmethod
64      def leftRotate(x):
65          y = x.right
66          x.right = y.left
67          y.left = x
68
69          Tree.calc(x)
70          Tree.calc(y)
71          return y
72
73      @staticmethod
74      def calc(node):
75          node.h = max(Tree.getHeight(node.left), Tree.getHeight(node.right)) + 1
76          node.size = Tree.getSize(node.left) + Tree.getSize(node.right) + 1
77          node.sum = Tree.getSum(node.left) + Tree.getSum(node.right) + node.key
78
79      @staticmethod
80      def getSum(node):
81          if not node:
82              return 0
83          return node.sum
84
85      @staticmethod
86      def getSize(node):
87          if not node:
88              return 0
89          return node.size
90
91      @staticmethod
92      def getHeight(node):
```

```
93          if not node:
94              return 0
95          return node.h
96
97
98  def main():
99      global tmp
100     n = int(stdin.readline())
101     a = list(map(int, stdin.readline().split()))
102
103     tree = Tree()
104
105     ans = [0] * n
106     for i in range(n - 1, -1, -1):
107         tmp = [0, 0]
108         tree.insert(a[i])
109         ans[i] = tmp[0] - tmp[1] * a[i]
110
111     res = [0] * n
112     stack = [0] * n
113     sz = 0
114     for i in range(n - 1, -1, -1):
115         while sz > 0 and a[stack[sz - 1]] < a[i]:
116             sz -= 1
117         if sz > 0:
118             res[i] = ans[stack[sz - 1]]
119         else:
120             res[i] = 0
121         stack[sz] = i
122         sz += 1
123
124     print(*res, sep=' ')
125
126
127 main()
```

سؤال ۱۱. برای این سوال از ساختار درخت min heap استفاده می‌کنیم. به این شکل که در آن k تا بالاترین نمره را نگه می‌داریم. هر نمره ای که اعلام می‌شود به هیپ اضافه می‌کنیم. هر وقت می‌خوابند k را کاهش می‌دهیم. در پایان هر درخواست هم بررسی می‌کنیم اگر سایز هیپ بزرگتر از k بود کوچکترین عدد هیپ که همان عدد ریشه‌ی آن است را از هیپ حذف می‌کنیم (و عدد یک برگ را در ریشه قرار داده سپس عمل bubble down را انجام می‌دهیم) چرا که این نمره دیگر هرگز جواب هیچ کدام از درخواست ها نخواهد بود و ازین پس همواره حداقل k نمره‌ی بزرگتر از آن وجود دارد.
پیچیدگی زمانی الگوریتم از $O((n+q)\log k)$ خواهد بود(البته می‌توان هیپ ابتدایی را با $O(n)$ هم ساخت).
کد نمونه (با تشکر از مهدی جعفری):

```
from sys import stdin, stdout

a = [-1.0]


def bubble_up(ind: int):
    while ind > 1 and a[ind] < a[ind // 2]:
        a[ind], a[ind // 2] = a[ind // 2], a[ind]
        ind //= 2


def bubble_down(ind: int):
    while True:
        L = ind * 2
        R = L + 1

        mn = (a[ind], ind)
        if L < len(a):
            mn = min(mn, (a[L], L))
        if R < len(a):
            mn = min(mn, (a[R], R))

        if mn[1] != ind:
            a[ind], a[mn[1]] = a[mn[1]], a[ind]
            ind = mn[1]
        else:
            break


def insert(val: float):
    a.append(val)
    bubble_up(len(a) - 1)

```

```
34
35 def remove():
36     a[1] = a[len(a) - 1]
37     a.pop()
38     bubble_down(1)
39
40
41 def main():
42     global a
43     n, m, k = map(int, stdin.readline().split())
44     a += list(map(float, stdin.readline().split()))
45     for i in range(n, 0, -1):
46         bubble_down(i)
47
48     for i in range(m):
49         tmp = list(map(float, stdin.readline().split()))
50         tmp[0] = int(tmp[0])
51         if tmp[0] == 1:
52             insert(tmp[1])
53         elif tmp[0] == 2:
54             while len(a) - 1 > k:
55                 remove()
56             stdout.write("%g" % a[1] + "\n")
57         else:
58             k -= 1
59 main()
```

سؤال ۱۲. نکته: یک رشته پرانتزگذاری معتبر است اگر و تنها اگر ویژگی زیر را داشته باشد:
اگر به ازای هر اندیس از این رشته، تعداد پرانتزبازها ")" از ابتدای رشته تا به اینجا بزرگتر یا مساوی تعداد پرانتزبسته‌ها "(" از ابتدای رشته تا به اینجا باشد و در نهایت در پایان رشته تعداد پرانتزبازها و بسته‌ها برابر باشد.
اثبات: اگر این شرط را نداشته باشیم که واضح است پرانتزگذاری معتبر نیست چراکه در یک پرانتزگذاری معتبر همواره ابتدا یک پرانتز باز می‌شود و سپس آن پرانتز را می‌بندیم و در نهایت هم همه پرانتز ها بسته می‌شوند. حال می‌ماند اثبات اینکه اگر این ویژگی را داشته باشیم حتما پرانتزگذاری معتبر است. اثبات این مورد هم با پشته‌ای که در ادامه برای حل سوال از آن کمک می‌گیریم قابل انجام است.
برای این سوال از یک پشته استفاده می‌کنیم. از سمت چپ رشته شروع میکنیم و اگر در حرف i ام باشیم، در صورتی که حرف iام ")" باشد i را به سر پشته اضافه می‌کنیم و اگر هم "(" بود سر پشته را حذف می‌کنیم.
حال برای بدست آوردن بزرگ‌ترین زیررشته‌ی معتبر، به ازای هر کدام از حروف رشته محاسبه می‌کنیم اگر انتهای زیررشته‌ی مورد نظر این حرف باشد بزرگترین بازه‌ی معتبر طولش چند است. اگر این طول از طول حداکثری که تا قبل از این بدست آورده بودیم بیشتر بود طول ماکسیمم را برابر طول جدید بدست آمده قرار می‌دهیم و تعداد زیررشته ها با طول ماکسیمم تا به این جا را برابر یک قرار می‌دهیم و اگر هم طول بدست آمده با بزرگترین طول بدست آمده پیش از این حرف برابر بود تعداد زیررشته‌های با طول ماکسیمم تا به این جای کار را هرچه بود یک واحد افزایش می‌دهیم.
برای بدست آوردن بزرگ‌ترین زیررشته ای که انتهایش به حرف iام ختم می‌شود به دو چیز نیاز داریم (توجه : فقط برای پرانتزبسته ها این مقدار را حساب میکنیم وگرنه این مقدار صفر است): اولین مورد، آخرین پرانتزبسته‌ای است که تا به اینجای کار به آن رسیده‌ایم و با پرانتزبازی match نشده (یعنی هنگامی که به آن رسیدیم پشته ما خالی بوده است) که اگر حرف iام که اکنون روی آن هستیم پرانتز بسته بود و پشته‌ی ما خالی بود این مقدار را برابر همین حرف iام قرار می‌دهیم و به حرف بعدی می‌رویم چرا که این پرانتز با پرانتزی match نمی‌شود و زیررشته‌ی معتبری شامل این حرف نداریم(برای اثبات از شرط گفته شده برای معتبر بودن پرانتزگذاری استفاده می‌کنیم). اما اگر پشته خالی نبود همان طور که گفتیم ابتدا سر آن را حذف می‌کنیم. سپس اگر پشته خالی نشد، ابتدای بزرگترین زیررشته‌ی معتبری که به حرف iام ختم می‌شود، حرف بعدیِ سرِ پشته‌ی کنونی است. چرا که در این بازه شرط گفته شده برای معتبر بودن رشته برقرار است. پس اگر حرف iام باشیم و عدد سر پشته (بعد از حذف سر قبلی) j باشد، زیررشته‌ی $s[j+1], s[j+2] \ldots, s[i]$ بزرگترین زیررشته معتبری است که انتهایش i است.
اما اگر پس از حذف سر پشته، پشته خالی شد، شروع بزرگترین بازه‌ی معتبری که به حرف iام ختم می‌شود، اندیس $p+1$ است به طوری که p اندیس آخرین جایی است که به پرانتز بسته‌ای رسیدیم که با چیزی match نشد و پشته خالی بود. زیرا در این زیررشته شرط معتبر بودن که گفته شد برقرار است. شروع این بازه قبل از اندیس $p+1$ هم نمی‌تواند باشد زیرا در غیر اینصورت باز هم با چک کردن شرط گفته شده می‌فهمیم شرط معتبر بودن برقرار نیست.
کد نمونه (با تشکر از حمیدرضا کلباسی) :

```
input()
a = input()

stack = []
i = 0
cnt = 0
mx = 0
```

```
pv = 1
for x in a:
  i += 1
  if (x == '('):
    stack.append(pv)
    pv = i + 1
  else:
    if len(stack) == 0:
      pv = i + 1
      continue
    j = stack.pop()
    pv = j
    l = i - j
    if l == mx:
      cnt += 1
    if l > mx:
      cnt = 1
      mx = l

if mx == 0:
  print(0, 1)
else:
  print(mx + 1, cnt)
```

موفق باشید